

INF560 / Unidad 2 / GL03

GUÍA DE LABORATORIO

Tailwind CSS

De CSS a utilidades

Desarrollo Web Backend • 5.º semestre • Gestión 02/2026

Laravel 13 • Tailwind CSS v4 • Duración: 1 sesión

Docente: M. Sc. Carrera de Ingeniería Informática • UATF

Proyecto acumulativo: [eventos-app](#)

00 Antes de empezar

En esta guía usted dará el primer paso del módulo de frontend de la materia: crear desde cero su proyecto Laravel 13 y estilizar sus vistas con Tailwind CSS v4. El objetivo no es memorizar clases, sino entender la idea central de Tailwind —el enfoque «utility-first»— construyendo piezas reales de la aplicación eventos-app que lo acompañará el resto del semestre.

DEFINICIÓN – utility-first

Tailwind no le da componentes ya hechos (como `.btn` o `.card`). Le da clases de utilidad de un solo propósito — una clase para el relleno, otra para el color, otra para la sombra— que usted combina directamente en el HTML. En lugar de escribir CSS en otro archivo, describe el diseño en el propio elemento.

Aunque INF560 es un curso de backend, todo backend web termina entregando vistas. En Laravel esas vistas son plantillas Blade, y Tailwind es la forma moderna y estándar de darles estilo. Dominarlo aquí le permitirá, más adelante, presentar sus CRUD y paneles con una interfaz profesional sin pelear con CSS.

NOTA – teoría breve, práctica larga

Esta guía es principalmente de ejercicios. Cada sección introduce solo el mínimo concepto necesario y enseguida pasa a construir. Escriba el código usted mismo; no copie y pegue sin leer.

01 Objetivos y prerequisites

Al terminar esta guía usted será capaz de:

- Crear un proyecto Laravel 13 nuevo y ponerlo en marcha con Tailwind v4 activo.
- Traducir reglas de CSS que ya conoce a sus utilidades equivalentes de Tailwind.
- Maquetar con Flexbox y Grid usando utilidades.
- Aplicar diseño responsive con el enfoque mobile-first.

Prerequisites (ya cubiertos en la GL01):

- PHP 8.3 o superior (*Laravel 13 lo exige*)
- Composer 2.x, Node.js 20+ y npm instalados.
- El instalador de Laravel disponible como comando `laravel`.

NOTA – sin base de datos

En esta guía no necesitamos base de datos: trabajaremos con datos de ejemplo definidos en un arreglo dentro de un controlador. Las migraciones y Eloquent llegarán en guías posteriores.

02 Ejercicio 1 — Crear el proyecto y verificar Tailwind v4

Ubíquese en la carpeta donde guarda sus proyectos y cree la aplicación. El instalador le hará algunas preguntas; responda como se indica.

```
terminal Copiar

# Crear el proyecto (elija las opciones indicadas abajo)
$ laravel new eventos-app

# Starter kit ..... None
# Testing framework .... Pest
# Database ..... SQLite (no la usaremos, pero evita configuración)
```

```
# Run npm install ..... Yes
```

Si el instalador no ejecutó npm por usted, entre a la carpeta e instálelo a mano:

```
terminal
```

[Copiar](#)

```
$ cd eventos-app
$ npm install
```

DEFINICIÓN – Tailwind ya viene incluido

Desde Laravel 12, un proyecto nuevo trae Tailwind CSS v4 preinstalado y cableado con Vite. No debe instalarlo ni crear tailwind.config.js: hacerlo manualmente suele romper la configuración por defecto.

Compruebe ese cableado. Abra dos archivos y confirme que se ven así (los de su proyecto pueden variar en detalles menores):

```
vite.config.js
```

[Copiar](#)

```
import { defineConfig } from 'vite';
import laravel from 'laravel-vite-plugin';
import { bunny } from 'laravel-vite-plugin/fonts';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [
    laravel({
      input: ['resources/css/app.css', 'resources/js/app.js'],
      refresh: true,
      fonts: [
        bunny('Instrument Sans', {
          weights: [400, 500, 600],
        }),
      ],
    }),
    tailwindcss(),
  ],
  server: {
    watch: {
      ignored: ['**/storage/framework/views/**'],
    },
  },
});
```

```
resources/css/app.css
```

[Copiar](#)

```
@import 'tailwindcss';

@source
'../../vendor/laravel/framework/src/Illuminate/Pagination/resources/views/*.blade.php';
@source '../../storage/framework/views/*.php';

@theme {
  --font-sans: 'Instrument Sans', ui-sans-serif, system-ui, sans-serif, 'Apple Color
Emoji', 'Segoe UI Emoji',
  'Segoe UI Symbol', 'Noto Color Emoji';
}
```

DEFINICIÓN – @source

La directiva @source le dice a Tailwind v4 en qué archivos buscar clases para generar únicamente el CSS que usted usa. Reemplaza al antiguo array content de tailwind.config.js.

Ahora levante el entorno de desarrollo. La forma más simple son dos terminales:

```
terminal 1 · servidor PHP
```

[Copiar](#)

```
$ php artisan serve
# Server running on http://127.0.0.1:8000
```

terminal 2 · compilador de assets (Vite)

Copiar

```
$ npm run dev
```

NOTA – un solo comando

Laravel 13 incluye un atajo que arranca todo a la vez (servidor, colas y Vite): `$ composer run dev`. Úselo si prefiere una sola terminal.

Verifique que Tailwind funciona. Cree su primera ruta y vista, y aplique una clase de utilidad.

GET `/eventos` → controlador → vista Blade

routes/web.php

Copiar

```
use App\Http\Controllers\EventoController;

Route::get('/eventos', [EventoController::class, 'index'])->name('eventos.index');
```

terminal · crear el controlador

Copiar

```
$ php artisan make:controller EventoController
```

app/Http/Controllers/EventoController.php

Copiar

```
class EventoController extends Controller
{
    public function index()
    {
        return view('eventos.index');
    }
}
```

Cree la vista `resources/views/eventos/index.blade.php` con un layout mínimo y una clase de prueba:

resources/views/eventos/index.blade.php

Copiar

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>eventos-app</title>
    @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>
<body class="bg-slate-50">
    <h1 class="text-3xl font-bold text-blue-600 p-8">
        Tailwind está funcionando
    </h1>
</body>
</html>
```

Abra `http://127.0.0.1:8000/eventos`. Si ve el título grande, en negrita y azul, Tailwind está funcionando. Ese es el punto de partida de todos los ejercicios siguientes.

ADVERTENCIA – el título se ve sin estilo

Si el texto aparece pequeño y negro, casi siempre es porque `npm run dev` no está corriendo o se detuvo. Manténgalo activo en su propia terminal mientras trabaja. Revise la sección «Problemas frecuentes» al final.

03 Ejercicio 2 — Card de evento: de CSS a utilidad

Aquí verá la traducción directa entre el CSS que ya conoce y las utilidades de Tailwind. Vamos a construir una tarjeta para un evento. Primero, el mapeo mental:

Propiedad CSS	CSS de siempre	Utilidad Tailwind
Relleno	<code>padding: 1.5rem</code>	<code>p-6</code>
Margen superior	<code>margin-top: 0.75rem</code>	<code>mt-3</code>
Fondo	<code>background: #fff</code>	<code>bg-white</code>
Color de texto	<code>color: #0f172a</code>	<code>text-slate-900</code>
Radio de borde	<code>border-radius: 0.75rem</code>	<code>rounded-xl</code>
Sombra	<code>box-shadow: 0 4px 6px...</code>	<code>shadow-md</code>
Tamaño de fuente	<code>font-size: 0.875rem</code>	<code>text-sm</code>
Peso de fuente	<code>font-weight: 700</code>	<code>font-bold</code>

Ahora reemplace el `<h1>` de prueba por esta tarjeta dentro del `<body>`:

```
resources/views/eventos/index.blade.php · <body> Copiar

<div class="min-h-screen flex items-center justify-center p-6">
  <div class="bg-white rounded-xl shadow-md p-6 max-w-sm">
    <span class="inline-block text-xs font-semibold text-blue-600
      bg-blue-50 px-2 py-1 rounded">Conferencia</span>

    <h3 class="mt-3 text-lg font-bold text-slate-900">
      Laravel Bolivia 2026</h3>
    <p class="mt-1 text-sm text-slate-500">
      Auditorio UATF · 20 de septiembre</p>
    <p class="mt-3 text-sm text-slate-600">
      Charlas sobre desarrollo web moderno con la comunidad local.</p>

    <button class="mt-4 w-full bg-blue-600 text-white
      text-sm font-medium py-2 rounded-lg">
      Ver detalles</button>
  </div>
</div>
```

EJEMPLO — lea la tarjeta como CSS

`bg-white rounded-xl shadow-md p-6` es exactamente «fondo blanco, esquinas redondeadas grandes, sombra media y relleno de 1.5rem». Cada palabra es una regla CSS. `max-w-sm` limita el ancho para que la tarjeta no se estire.

Experimente para fijar el concepto: cambie `shadow-md` por `shadow-lg`, `rounded-xl` por `rounded-none`, y `bg-blue-600` por `bg-cyan-600`. Guarde y observe el navegador recargarse solo. Así se siente el flujo utility-first.

04 Ejercicio 3 — Navbar responsive con Flexbox

Flexbox alinea elementos en una fila o columna. En Tailwind se activa con la clase `flex`, y se controla con utilidades como `items-center` (alineación vertical) y `justify-between` (separación horizontal). Añada esta barra de navegación al inicio del `<body>`, antes de la tarjeta:

```
resources/views/eventos/index.blade.php · navbar Copiar
```

```
<nav class="bg-white border-b border-slate-200">
  <div class="max-w-6xl mx-auto px-4 h-16
    flex items-center justify-between">
    <a href="#" class="text-lg font-bold text-slate-900">eventos-app</a>

    <div class="hidden sm:flex items-center gap-6 text-sm text-slate-600">
      <a href="#" class="hover:text-blue-600">Inicio</a>
      <a href="#" class="hover:text-blue-600">Eventos</a>
      <a href="#" class="hover:text-blue-600">Acerca</a>
    </div>

    <button class="sm:hidden text-slate-600">Menú</button>
  </div>
</nav>
```

DEFINICIÓN – el trío de Flexbox

flex → convierte el contenedor en flexible (fila por defecto).

items-center → centra los hijos en el eje vertical.

justify-between → empuja el primer y último hijo a los extremos.

Fíjese en `hidden sm:flex` y `sm:hidden`: ese es el primer contacto con responsive. En pantallas pequeñas el menú de enlaces se oculta (`hidden`) y aparece el botón «Menú»; a partir del breakpoint `sm` el menú se muestra (`sm:flex`) y el botón se oculta. Achique la ventana del navegador para verlo.

NOTA – los estados vienen después

`hover:text-blue-600` es un adelanto de los estados (`hover`, `focus`, etc.). Los trabajaremos a fondo en la GL04; por ahora basta con notar que el enlace cambia de color al pasar el cursor.

05 Ejercicio 4 — Grid de eventos responsive

CSS Grid organiza contenido en columnas y filas. Con Tailwind, `grid` activa la rejilla y `grid-cols-N` fija el número de columnas. Combinado con breakpoints, logramos un diseño que se adapta: una columna en el teléfono, dos en la tableta y tres en el escritorio.

DEFINICIÓN – mobile-first

En Tailwind, una clase sin prefijo (`grid-cols-1`) aplica a TODOS los tamaños, empezando por el más pequeño. Los prefijos `sm:` `md:` `lg:` solo AÑADEN cambios para pantallas más grandes. Por eso se diseña «primero para móvil».

Para tener varias tarjetas necesitamos datos. Como no usamos base de datos, defínalos en el controlador y páselos a la vista:

```
app/Http/Controllers/EventoController.php

public function index()
{
    $eventos = [
        ['titulo' => 'Laravel Bolivia 2026', 'tipo' => 'Conferencia',
        'lugar' => 'Auditorio UATF', 'fecha' => '20 de septiembre'],
        ['titulo' => 'Taller de Tailwind v4', 'tipo' => 'Taller',
        'lugar' => 'Laboratorio 3', 'fecha' => '27 de septiembre'],
        ['titulo' => 'Hackathon UATF', 'tipo' => 'Competencia',
        'lugar' => 'Campus Central', 'fecha' => '4 de octubre'],
    ];

    return view('eventos.index', compact('eventos'));
}
```

Copiar

En la vista, envuelva las tarjetas en una rejilla y recórralas con `@foreach`. Reemplace la tarjeta única del Ejercicio 2 por este bloque (debajo del navbar):

resources/views/eventos/index.blade.php · grid

Copiar

```
<div class="max-w-6xl mx-auto px-4 py-8">
  <div class="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6">
    @foreach ($eventos as $evento)
      <div class="bg-white rounded-xl shadow-md p-6">
        <span class="inline-block text-xs font-semibold
          text-blue-600 bg-blue-50 px-2 py-1 rounded">
          {{ $evento['tipo'] }}</span>
        <h3 class="mt-3 text-lg font-bold text-slate-900">
          {{ $evento['titulo'] }}</h3>
        <p class="mt-1 text-sm text-slate-500">
          {{ $evento['lugar'] }} · {{ $evento['fecha'] }}</p>
      </div>
    @endforeach
  </div>
</div>
```

EJEMPLO – descifre la rejilla

`grid-cols-1 sm:grid-cols-2 lg:grid-cols-3` significa: 1 columna en móvil, 2 desde tabletas (sm) y 3 desde escritorio (lg). `gap-6` es el espacio entre tarjetas. Redimensione el navegador y véalo reacomodarse.

06 Ejercicio 5 — Estados y transiciones

Un estado es una condición temporal de un elemento: el cursor encima, el foco del teclado, el clic sostenido, o estar deshabilitado. En Tailwind se activan con un prefijo antes de la utilidad: `hover:`, `focus:`, `active:`, `disabled:`.

DEFINICIÓN – prefijo de estado

`hover:bg-blue-700` significa «cuando el cursor esté encima, el fondo pasa a blue-700». La utilidad base (`bg-blue-600`) sigue siendo el estado normal; el prefijo solo añade el cambio.

Tome el botón «Ver detalles» de su tarjeta y dэле vida. Reemplace su clase por esta:

botón con estados

Copiar

```
<button class="bg-blue-600 text-white text-sm font-medium px-4 py-2 rounded-lg
  transition
  hover:bg-blue-700
  active:bg-blue-800
  focus:outline-none focus:ring-2 focus:ring-blue-300
  disabled:opacity-50 disabled:cursor-not-allowed">
  Ver detalles
</button>
```

EJEMPLO – lea cada estado

`transition` suaviza el cambio; `hover:` al pasar el cursor; `active:` mientras se mantiene presionado; `focus:ring` dibuja un anillo al enfocar con el teclado (Tab); `disabled:` aplica cuando el botón tiene el atributo `disabled`.

Compruébelo: pase el cursor, presione y suéltelo, y navegue con Tab. Luego agregue el atributo `disabled` al `<button>` para ver el estado deshabilitado; quítelo al terminar.

Aplique lo mismo a los enlaces del navbar para que la transición de color sea suave:

navbar · enlaces

Copiar

```
<a href="#" class="text-slate-600 transition-colors
```

```
hover:text-blue-600 focus:text-blue-600">Eventos</a>
```

07 Ejercicio 6 — Interacción con group y peer

A veces el estado de un elemento debe afectar a OTRO. Para eso existen dos modificadores clave, y ninguno necesita JavaScript.

DEFINICIÓN – group y peer

group: marca un contenedor padre; sus hijos reaccionan con group-hover:, group-focus:, etc.

peer: marca un elemento hermano; los que vienen después reaccionan con peer-focus:, peer-checked:, etc.

Convierta la tarjeta entera en un enlace que reacciona en conjunto al pasar el cursor. Note la clase group en el padre y group-hover: en los hijos:

tarjeta interactiva con group

Copiar

```
<a href="#" class="group block bg-white rounded-xl shadow-md p-6
  transition hover:shadow-lg">
  <h3 class="text-lg font-bold text-slate-900
    transition-colors group-hover:text-blue-600">
    Laravel Bolivia 2026</h3>
  <p class="mt-1 text-sm text-slate-500">Auditorio UATF</p>
  <span class="mt-3 inline-block text-sm text-slate-400
    group-hover:text-blue-500">Ver más &rarr;</span>
</a>
```

EJEMPLO – un solo hover, varios cambios

Al pasar el cursor por CUALQUIER parte de la tarjeta, sube la sombra (hover:shadow-lg del propio padre) y, a la vez, el título y el enlace «Ver más» cambian de color (group-hover:). Un único gesto, varias reacciones coordinadas.

Ahora peer, con un pequeño formulario de suscripción. El mensaje de ayuda reacciona al foco del campo, aunque son elementos separados:

campo con peer

Copiar

```
<div class="max-w-sm">
  <input type="email" placeholder="correo@uatf.edu.bo"
    class="peer w-full border border-slate-300 rounded-lg px-3 py-2
      focus:border-blue-500 focus:outline-none">

  <p class="mt-1 text-xs text-slate-400 peer-focus:text-blue-600">
    Le enviaremos la confirmación a este correo.</p>
</div>
```

NOTA – el orden importa en peer

peer-focus: solo funciona en elementos que aparecen DESPUÉS del elemento con la clase peer en el HTML. Por eso el <p> va después del <input>.

08 Trabajo independiente

Ahora le toca a usted. Cree una página de práctica que reúna todo lo aprendido, sin copiar los ejercicios anteriores tal cual.

GET [/practica-ui](#) → su propia página

Requisitos:

- Agregue la ruta GET /practica-ui apuntando a un método suyo (por ejemplo practica) en EventoController.
- Incluya el navbar responsive del Ejercicio 3 en la parte superior.
- Muestre al menos 5 eventos (arreglo en el controlador) en una rejilla responsive de 1 / 2 / 3 columnas.
- Cada tarjeta debe usar spacing, color, tipografía, radio y sombra con utilidades de Tailwind.
- Añada al menos una variación propia respecto a los ejemplos (otro color de acento, otra sombra, un botón, etc.).

NOTA – se evalúa el criterio, no la copia

La página debe funcionar y verse ordenada y responsive. Si redimensiona el navegador y el contenido se reacomoda con elegancia, va por buen camino.

07 Checklist de entrega

- ☐ El proyecto eventos-app se crea y el servidor responde en /eventos.
- ☐ npm run dev corre sin errores y Tailwind v4 aplica estilos.
- ☐ La tarjeta de evento usa relleno, color, radio y sombra correctos.
- ☐ El navbar se adapta: menú en escritorio, botón en móvil.
- ☐ La rejilla muestra 1 / 2 / 3 columnas según el tamaño de pantalla.
- ☐ La página /practica-ui cumple todos los requisitos del trabajo independiente.
- ☐ El proyecto está versionado con Git y subido a su repositorio del semestre.

08 Problemas frecuentes

Síntoma	Causa y solución
Las clases de Tailwind no aplican	npm run dev no está corriendo, o la clase está en un archivo no cubierto por @source. Reinicie el compilador y confirme que la vista es un .blade.php dentro de resources/views.
Unable to locate file in Vite manifest	Falta compilar los assets. Ejecute npm run dev (desarrollo) o npm run build (producción). Revise que @vite apunte a resources/css/app.css.
El estilo se ve solo la primera vez y luego no	Cerró la terminal de Vite. Mantenga npm run dev activo mientras edita las vistas.
Address already in use (puerto 8000)	Ya hay un servidor corriendo. Cíérrelo, o use otro puerto: php artisan serve --port=8001.
Instalé Tailwind «a mano» y dejó de funcionar	Laravel 13 ya lo trae. No cree tailwind.config.js ni instale PostCSS. Si lo hizo, revierta esos cambios y conserve solo el plugin @tailwindcss/vite.

09 Rúbrica de evaluación (100 pts)

Criterio	Pts
Proyecto Laravel 13 creado y servidor corriendo con Tailwind v4 activo	20
Card de evento con spacing, color, tipografía, radio y sombra correctos	20
Navbar responsive con Flexbox (menú/botón según tamaño)	15
Grid de eventos responsive (1 / 2 / 3 columnas)	20

Criterio	Pts
Trabajo independiente /practica-ui funcional y con variación propia	20
Código ordenado, indentado y versionado en Git	5
Total	100

Fin de la GL03. Continúa en la GL04 — Tailwind CSS II: estados, interacción y componentización.